

Introduction

If you've ever wondered why the same website loads instantly sometimes and crawls other times, the answer usually isn't your internet connection — it's caching. Specifically, it's the difference between a cache hit and a cache miss on a CDN (Content Delivery Network).

This topic shows up constantly in system design interviews because it tests something deeper than trivia about CDNs. It tests whether you understand the fundamental tradeoff behind every caching system ever built: you can't store everything everywhere, so you have to accept that someone, somewhere, pays the cost of a miss. Once you understand that tradeoff at the CDN level, you'll recognize it in browser caches, database query caches, and application-level caches too. It's one idea wearing a lot of different hats.

Problem Overview

Here's the scenario: a user clicks a link to load a web page or asset — an image, a script, a video segment, doesn't matter. That request doesn't go straight to the company's main server. Instead, it lands on the nearest CDN edge server, a machine geographically close to the user, whose entire job is to serve content fast by keeping local copies of files.

The edge server checks its local cache. Two outcomes are possible:

- **Cache hit** — the file is already there. It gets served immediately.
- **Cache miss** — the file isn't there. The edge server has to go get it from the origin server, the actual source of truth, which might be thousands of miles away.

The question this problem is really asking is: what happens during that miss, why can't it be avoided entirely, and why doesn't the "obvious" fix (just cache everything everywhere) work?

Example

Imagine a product page on an e-commerce site. Nobody in Tokyo has requested that specific page in the last six hours. The nearest edge server to Tokyo has since evicted or never stored that file.

A user in Tokyo clicks the link.

1. Request hits the Tokyo edge server.

2. Edge checks its cache — not found.
3. Edge forwards the request to the origin server (say, based in the US).
4. Origin builds/returns the response.
5. Response travels back across the network to the Tokyo edge.
6. Edge serves it to the user **and** stores a copy locally.

That entire round trip — Tokyo to origin and back — is why this particular request feels slow. Thirty seconds later, a second user in Tokyo clicks the same link. This time the edge already has the file. Step 2 succeeds immediately. No trip to origin. The response is close to instant.

Same link, same content, two completely different experiences — because of timing, not luck.

Intuition

The mental model that makes this click: an edge server is not a source of truth, it's a local convenience copy. Think of a public library branch versus a central archive. Your neighborhood branch doesn't have every book ever published — it has what people nearby actually ask for. If you request something rare, the branch has to send away for it. Once it arrives, they keep a copy on the shelf, because odds are somebody else in the neighborhood will want it soon too.

An experienced engineer reasoning about this system asks two questions in order:

1. **Where does data live closest to the requester, and what happens when it's not there?** That's the miss path — go get it from the authoritative source.
2. **What do you do with the response once you have it, so the next request doesn't pay the same cost?** That's the store-on-the-way-back step, which turns future misses into hits.

The tradeoff falls out naturally once you accept a constraint: storage at the edge is limited, and there are thousands of edge locations. You cannot mirror the entire internet at every location. So the system has to be reactive — it learns what's popular in a given region by watching what gets requested, and it caches based on actual demand rather than trying to predict everything in advance.

Brute Force Solution

The "brute force" approach to this design problem is: **replicate everything, everywhere, all the time.**

Idea: Every edge server holds a full copy of every file that exists on the origin. No cache miss is ever possible because nothing is ever missing.

Algorithm: - On any content update at the origin, push a copy to every edge location immediately. - Every read request is served locally, no exceptions.

Advantages: - Zero latency from the "miss" path — every request is a hit. - Simple mental model, no eviction logic needed.

Disadvantages: - Storage cost scales with (number of edge locations) $\times$ (total size of all content), which is enormous and largely wasted, since most content isn't requested at most locations. - Every write requires propagating data globally before it's even been asked for, adding write-side latency and network cost for content nobody may ever request in that region. - Doesn't scale as content volume grows; the internet's total data size vastly exceeds any realistic edge storage budget.

Time complexity: $O(1)$ per read (always a hit), but $O(E)$ per write, where E is the number of edge locations — you pay the propagation cost for every single edge, every single time, regardless of demand.

Space complexity: $O(E \times D)$, where D is total data size — the killer constraint that makes this approach impossible at real-world scale.

Optimal Solution

The optimal approach is **demand-driven caching**: only store a file at an edge location after someone at that location has actually requested it, and let that action define what deserves a spot in the cache.

Here's the flow, step by step:

Step	What happens
1	Request arrives at nearest edge server
2	Edge checks local cache for the file
3a	Hit: serve immediately from edge
3b	Miss: edge forwards request to origin
4	Origin builds/retrieves the response
5	Response travels back to edge
6	Edge serves response to user and stores a copy locally
7	Future requests for that file at this edge become hits

The elegance here is that the system doesn't need to know in advance what's popular where. Each cache miss is simultaneously a failure for the requester (they pay the round trip) and useful signal for the system (this file matters here, keep it around). The cost of the miss is paid once, and every subsequent request in that region benefits.

Most CDNs layer additional logic on top of this — TTLs (time-to-live) that expire stale entries, LRU/LFU eviction when storage fills up, and cache invalidation when the origin content changes — but the round-trip-then-store pattern above is the core mechanism underneath all of it.

Python Code

While CDNs are distributed infrastructure, not a single Python program, the core mechanism — check cache, fall back to origin, store on the way back — is exactly the same pattern used in a local caching layer. Here's a minimal, faithful simulation:

```
import time

class Origin:
    """Simulates the authoritative source server."""

    def fetch(self, key: str) -> str:
        # Simulate real network + processing latency to a distant origin.
        time.sleep(0.3)
        return f"content-for-{key}"

class EdgeCache:
    """Simulates a single CDN edge node."""

    def __init__(self, origin: Origin):
        self.origin = origin
        self.store: dict[str, str] = {}

    def get(self, key: str) -> tuple[str, bool]:
        """Return (content, was_cache_hit)."""
        if key in self.store:
            return self.store[key], True

        # Cache miss: full round trip to origin.
        content = self.origin.fetch(key)
        self.store[key] = content
        return content, False

def demo() -> None:
    edge = EdgeCache(Origin())

    # First request: cache miss, pays the round trip.
    content, hit = edge.get("product-page-42")
    assert hit is False
    assert content == "content-for-product-page-42"
```

```
# Second request for the same key: cache hit, no round trip.
content, hit = edge.get("product-page-42")
assert hit is True
assert content == "content-for-product-page-42"

print("cache miss then hit behaves correctly")

if __name__ == "__main__":
    demo()
```

Code Walkthrough

- `Origin.fetch` stands in for the real backend server, deliberately slow (`time.sleep(0.3)`) to represent network distance and processing time.
- `EdgeCache.store` is a plain dictionary representing the edge node's local cache — in a real CDN this would be a distributed key-value store with eviction policies, but the interface is the same.
- `EdgeCache.get` is the entire algorithm in five lines: check local store, return early on hit, otherwise fetch from origin, save it, then return it.
- The `bool` return value exists purely so callers (and the demo) can observe whether a given request was a hit or a miss — real CDNs expose this as response headers like `X-Cache: HIT` or `MISS`.

Dry Run

Request 1: `edge.get("product-page-42")` - `"product-page-42"` not in `self.store` → miss. - Calls `origin.fetch(...)`, waits ~0.3s, gets `"content-for-product-page-42"`. - Stores it in `self.store["product-page-42"]`. - Returns `(content, False)`.

Request 2: `edge.get("product-page-42")` - `"product-page-42"` **is** in `self.store` → hit. - Returns immediately, no call to `origin.fetch`, no delay. - Returns `(content, True)`.

This mirrors exactly what happened with the two Tokyo visitors: the first pays the origin round trip, the second gets an instant response because the first one already did the work.

Complexity Analysis

- **Time:** $O(1)$ for a cache hit (dictionary lookup). A cache miss costs $O(1)$ locally plus the network round-trip time to the origin, which in practice dominates — this is why misses feel slow and hits

feel instant.

- **Space:** $O(k)$ at each edge, where k is the number of distinct keys currently cached at that location — bounded by the edge's storage capacity, not by the total size of all content that exists.
- These bounds are correct because the cache never grows past what's actually been requested at that location, and lookups are direct key access rather than any kind of search.

Alternative Solutions

Push-based pre-warming: instead of waiting for a miss, predict likely-popular content (based on trending signals, scheduled releases, or known traffic patterns) and push it to edges in advance. This eliminates the *first* miss for predicted content, but adds prediction complexity and still can't cover the long tail of rarely-requested files — those still rely on demand-driven caching. In practice, CDNs often combine both: predictive pre-warming for known high-traffic events, and lazy demand-driven caching for everything else.

Edge Cases

- **Empty cache (cold edge node):** every request is a miss until the cache warms up — this is why a brand-new edge location, or one after a cache flush, temporarily behaves slower for everyone.
- **Duplicate near-simultaneous requests for the same missing key:** without request coalescing, two users hitting a cold cache at nearly the same moment can both trigger separate origin fetches ("cache stampede"); real CDNs typically lock or queue the second request behind the first.
- **Content that changes at the origin:** a stale cached copy can be served after the source data has updated, unless TTLs or invalidation are in place.
- **Extremely large files:** may exceed practical caching thresholds and get treated differently (e.g., streamed instead of fully cached).
- **Content marked non-cacheable** (personalized responses, authenticated pages): should bypass the cache entirely and always hit the origin — caching these by mistake can leak one user's data to another.

Common Mistakes

1. Assuming a cache miss means something is broken — it's expected, normal behavior for infrequently requested content.
2. Believing "cache everything everywhere" is a real fix, without accounting for storage cost at scale.

3. Forgetting to store the response after fetching it from origin, which turns every future request into a miss again.
4. Ignoring TTL/invalidation, leading to stale content being served indefinitely.
5. Not handling concurrent misses for the same key, causing duplicate origin load (thundering herd/cache stampede).
6. Treating CDN caching as unrelated to browser caching or database caching, when they're the same underlying pattern.

Interview Questions

- How would you handle a cache stampede when many users request the same missing key simultaneously?
- How do you decide what TTL to assign to different types of content?
- How would you invalidate a cached file across all edge locations when the origin content changes?
- What's the difference between cache-aside, write-through, and write-back caching strategies?
- How would you measure and improve a CDN's cache hit ratio?

Similar Problems

- **LRU Cache (LeetCode 146):** implementing the eviction policy an edge server needs once storage fills up.
- **Design a Rate Limiter:** another system design problem centered on tracking and reacting to per-key state under load.
- **Design a Key-Value Store:** the data-structure backbone behind any cache implementation.
- **Two Sum (as a warm-up for hash map fluency):** the same $O(1)$ lookup principle that makes cache hits fast underlies this classic problem.

Key Takeaways

A cache miss isn't a failure — it's the mechanism by which a cache learns what's worth storing. The edge-to-origin round trip is the price paid once so that every subsequent nearby request becomes instant. Trying to eliminate misses entirely by caching everything everywhere doesn't scale; the real solution is accepting that misses happen and making sure each one pays for itself by warming the cache for the next request. This same pattern — check local, fall back to source, store on the way back — shows up in browser caches, database query caches, and any layered system with a fast local tier and a slow authoritative source.

Watch the Video

For the full visual walkthrough of this concept, including the Tokyo example animated step by step, watch the video here: {LINK}

About the Series

This article is part of **Fun with Learning Technology**, a series that takes concepts that feel like backend magic — caching, networking, distributed systems, the stuff that quietly runs the internet — and breaks them down piece by piece until they're intuitive rather than intimidating.

Call To Action

If this helped the concept click, give the video a like on YouTube and subscribe for more breakdowns like this one. Subscribe to the Substack newsletter for deeper written explainers that go beyond what fits in a video. Drop a comment with a topic you want broken down next, and share this with a fellow developer who's still cursing at that spinner.

Quick Review

CDN cache miss: what's the tell that a request took this path?

Extra latency/round trip to origin plus a MISS header — edge had no copy so it fetched, served, and cached it.

Time cost of a cache miss vs a hit?

Miss: origin round trip (~edge-to-origin RTT + origin processing). Hit: single edge lookup, no origin call.

Space tradeoff a CDN makes on cache misses?

Caches response at the edge after fetching, trading storage capacity for fewer future origin round trips.

Trickiest bug: naive fix for 'edge is stuck' on a miss?

Don't let every miss hit origin directly — concurrent misses for the same key can stampede origin; coalesce requests instead.

Cache miss vs cache hit — key distinguishing behavior?

Hit serves straight from edge store; miss requires origin round trip before serving and storing the response.

Interview follow-up: how prevent 'obvious fix doesn't scale' issue?

Use request coalescing/locking so only one origin fetch happens per key during simultaneous misses, others wait and reuse it.

How a CDN Serves a Cache Miss — free Python cheat sheet